

Методы обработки естественного языка



Можно ли научить машины извлекать из текстов важные данные?

Этой проблемой занимается особое направление искусственного интеллекта:

обработка естественного языка, или NLP (Natural Language Processing).

Компьютеры не могут в полной мере понимать человеческий язык, однако они на многое способны. NLP может делать по-настоящему волшебные вещи и экономить огромное количество времени.

Natural Language Processing (NLP) – обработка естественного языка – подраздел информатики и искусственного интеллекта, посвященный тому, как компьютеры анализируют естественные (человеческие) языки.

NLP позволяет применять алгоритмы машинного обучения для текста и речи.

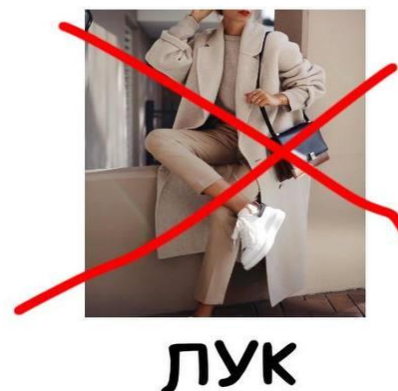
Например, можно использовать NLP, чтобы создавать системы вроде распознавания речи, обобщения документов, машинного перевода, выявления спама, распознавания именованных сущностей, ответов на вопросы, автокомплита, предиктивного ввода текста и т.д.

Задачи NLP

- Машинный перевод.
- Анализ тональности текста (*классификация отзывов на положительные, отрицательные и нейтральные*)
- Спам фильтрация
- Определение фейковых новостей (*здесь анализ текста выступает как часть общего алгоритма*)
- Многоклассовая классификация новостей по категориям (рубрикация)
- Извлечение именованных сущностей, NER (Named-entity recognition)
- Диалоговые системы, чат-боты
- Саммаризация (*на вход система принимает текст большого размера, а выходом служит текст меньшего размера: пересказ, аннотация*)
- Argumentation mining, поиск обоснования в тексте (*даны факт и текст, нужно найти обоснование данного факта в тексте*)

Почему решать задачи NLP сложно?

- **Полисемия** (многозначные слова имеют общий исходный смысл)
- **Омонимия** (разные по смыслу слова произносятся и пишутся одинаково)
- Данные, «хранящиеся» в форме текста на естественном языке, обладают неопределенностью – человек понимает значение того или иного слова, словосочетания, предложения **исходя из контекста**.



NLP пайплайн шаг за шагом

[NLP – это весело! Обработка естественного языка на Python \(proglib.io\)](https://proglib.io)

Итак, разобьем процесс языкового анализа на стадии:

London is the capital and most populous city of England and the United Kingdom. Standing on the River Thames in the south east of the island of Great Britain, London has been a major settlement for two millennia. It was founded by the Romans, who named it Londinium.

В этом параграфе содержится несколько полезных фактов. Хотелось бы, чтобы компьютер смог понять, что Лондон – это город, что он расположен в Англии, был основан римлянами и т. д. Но прежде всего мы должны научить его самым базовым концепциям письменного языка.

Шаг 1. Выделение предложений

Первый этап пайплайна – разбить текст на отдельные предложения. В результате получим следующее:

- London is the capital and most populous city of England and the United Kingdom.
- Standing on the River Thames in the south east of the island of Great Britain, London has been a major settlement for two millennia.
- It was founded by the Romans, who named it Londinium.

Можно предположить, что каждое предложение – это самостоятельная мысль или идея.

Шаг 2. Токенизация, или выделение слов

Следующий шаг конвейера – выделение отдельных слов или токенов – токенизация. Результат токенизации первого предложения выглядит так:

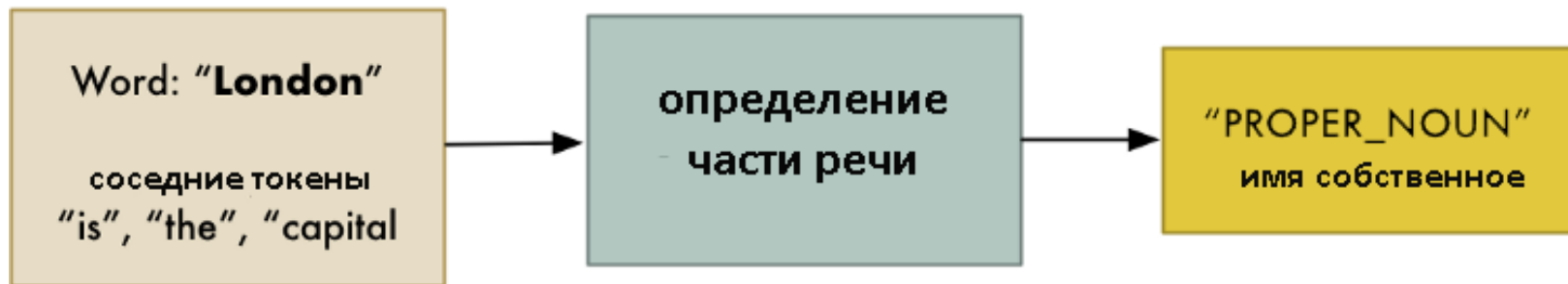
«London», «is», «the», «capital», «and», «most», «populous», «city», «of», «England», «and», «the», «United», «Kingdom», «.»»

В английском языке это несложно. Мы просто отделяем фрагмент текста каждый раз, когда встречаем пробел. Знаки препинания тоже являются токенами, поскольку могут иметь важное значение.

Шаг 3. Определение частей речи

Теперь посмотрим на каждый токен и **постараемся угадать, какой частью речи он является: существительным, глаголом, прилагательным** или чем-то другим. Зная роль каждого слова в предложении, можно понять его общий смысл.

На этом шаге **мы будем анализировать каждое слово вместе с его ближайшим окружением** с помощью предварительно подготовленной **классификационной модели**:



Эта модель была обучена на миллионах английских предложений с уже обозначенными частями речи и теперь способна их распознавать. Этот анализ основан на статистике – на самом деле модель не понимает смысла слов, вложенного в них человеком. Она просто знает, как угадать часть речи, основываясь на похожей структуре предложений и ранее изученных токенах.

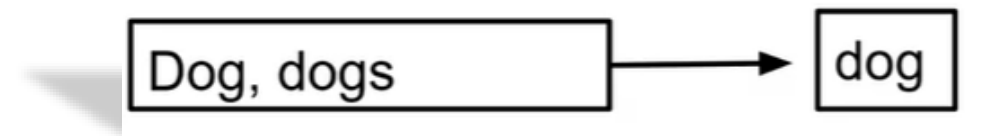


С этой информацией уже можно начинать анализировать смысл. Например, мы видим существительные «London» и «capital», вероятно, в предложении говорится о Лондоне.

Шаг 4. Нормализация токенов

Нормализовать токены можно с помощью **стемминга** и **лемматизации**

Token normalization



Стемминг – это грубый эвристический процесс, который отрезает «лишнее» от корня слов, часто это приводит к потере словообразовательных суффиксов.

Лемматизация – это более тонкий процесс, который использует словарь и морфологический анализ, чтобы в итоге привести слово к его канонической форме – лемме.

Стеммер (конкретная реализация алгоритма стемминга) действует без знания контекста и, соответственно, не понимает разницу между словами, которые имеют разный смысл в зависимости от части речи. Однако у стеммеров есть и свои преимущества: их проще внедрить и они работают быстрее.

Примеры:

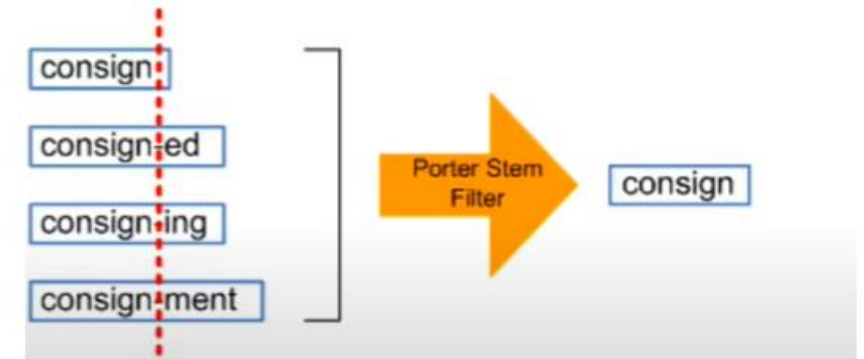
1. Слово good – это лемма для слова better. Стеммер не увидит эту связь, так как здесь нужно сверяться со словарем.
2. Слово play – это базовая форма слова playing. Тут справятся и стемминг, и лемматизация.
3. Слово meeting может быть как нормальной формой существительного, так и формой глагола to meet, в зависимости от контекста. В отличие от стемминга, лемматизация попыбует выбрать правильную лемму, опираясь на контекст.

Стемминг

Porter's stemmer:

- **Heuristics, applied one-by-one:**
 - SSES - SS (dresses - dress)
 - IES - I (ponies - poni)
 - S - <empty> (dogs - dog)
- **What's wrong?**
 - **Overstemming and understemming**

Stemming: removing and replacing suffixes to get to the root of the word (**stem**)

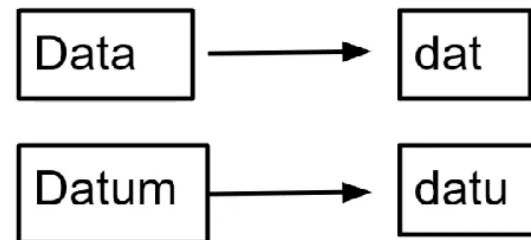


Проблеммы стемминга

Overstemming

- University
 - Universal
 - Universities
 - Universe
- } Univers

Understemming



Лемматизация

В английском и большинстве других языков слова могут иметь различные формы:

- I had a pony.
- I had two ponies.

*Оба предложения содержат существительное «pony», но с разными окончаниями. Компьютер должен **знать основную форму каждого слова**, чтобы понимать, что речь идет об одной и той же концепции пони. Иначе токены «pony» и «ponies» будут восприняты как совершенно разные.*

В NLP этот процесс называется **лемматизацией** – нахождением основной формы (леммы) каждого слова в предложении. Лемматизация обычно выполняется простым поиском форм в таблице.



Список лемматизации

глаз	глаза	тут	туда
глаз	глазу	тут	туту
глаз	глазом	тут	тутом
глаз	глазе	тут	туте
глаз	глазам	тут	туты
глаз	глазами	тут	тутов
глаз	глазах	тут	тутам

Глаголы можно привести к неопределенной форме.

Таким образом, предложение «I **had** two **ponies**» превращается в «I [**have**] two [**pony**]».

Вот так выглядит наше предложение после обработки:

London	is	the	capital	and	most	populous ...
	be					
Имя собственное	Глагол	Артикль	Существительное	Союз	Наречие	Прилагательное

Шаг 5. Определение стоп-слов

В английском очень много вспомогательных слов, например, «and», «the», «a». При статистическом анализе текста эти токены создают много шума, так как появляются чаще, чем остальные. *Некоторые NLP пайплайны отмечают их как стоп-слова и отсеивают перед подсчетом количества.*

Теперь наше предложение выглядит следующим образом:



Для обнаружения стоп-слов обычно используются готовые таблицы. Однако нет единого стандартного списка, подходящего в любой ситуации. Игнорируемые токены могут меняться, все зависит от особенностей проекта.

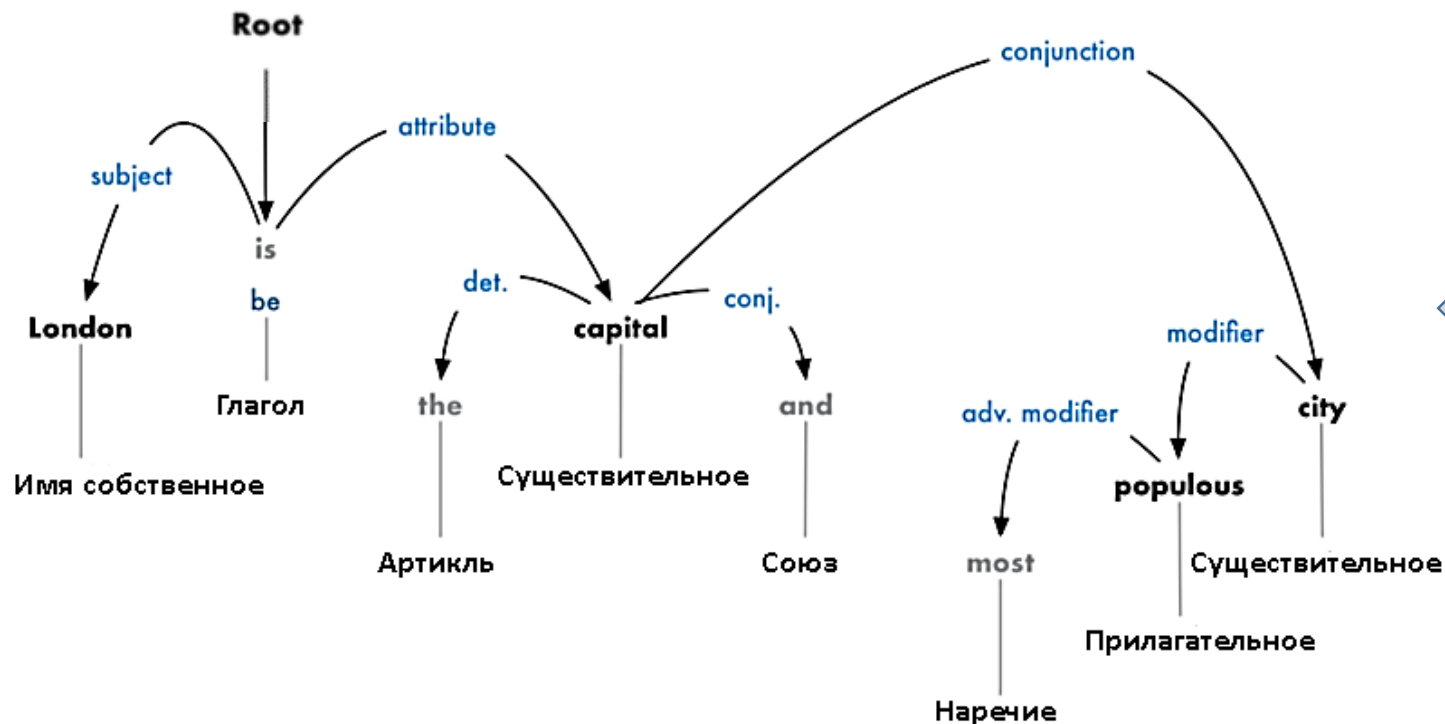
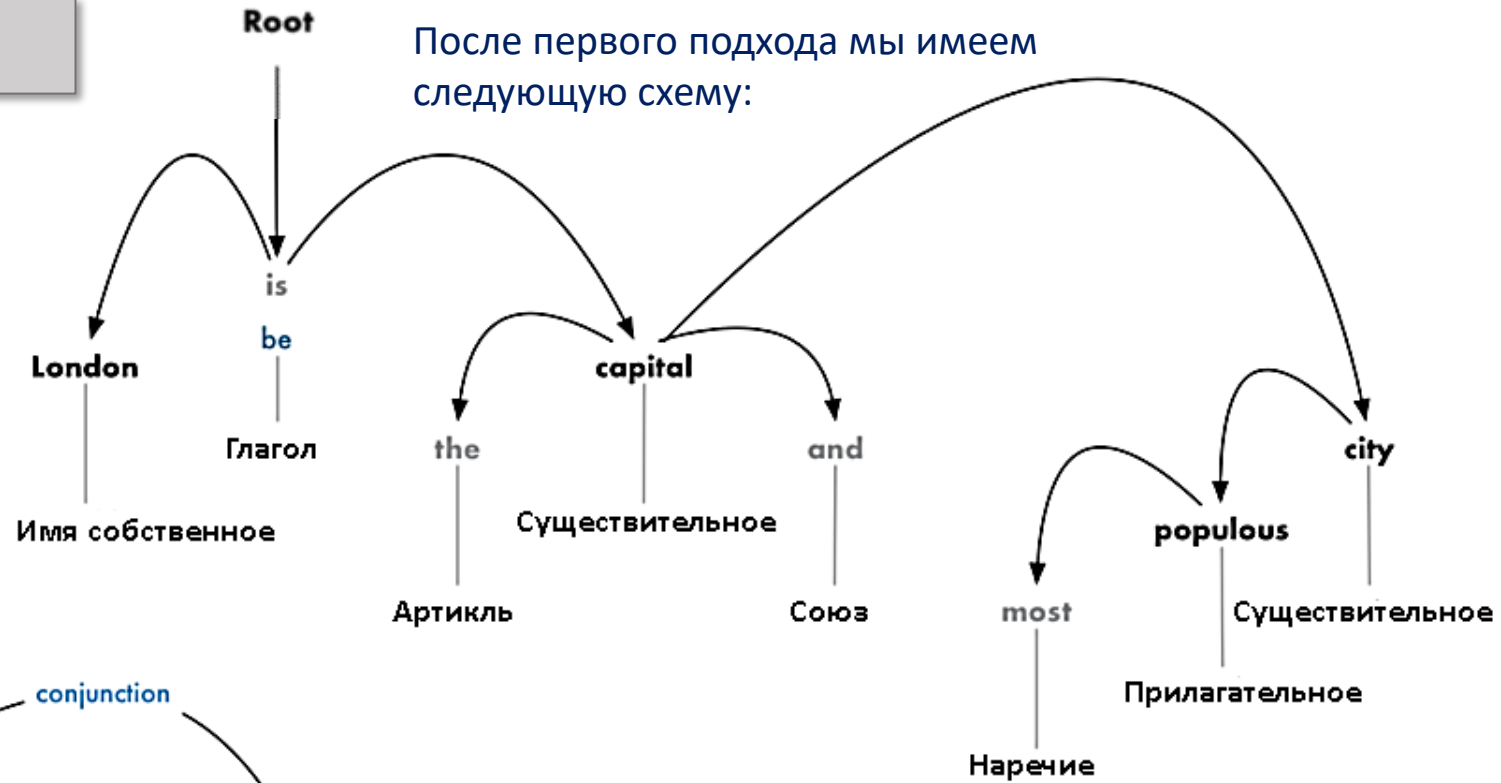
Например, если вы решите создать движок для поиска рок-групп, вероятно, вы не станете игнорировать артикль «the». Он встречается в названии множества коллективов, а одна известная группа 80-х даже называется «The The!».

Шаг 6. Парсинг зависимостей

Теперь необходимо установить взаимосвязь между словами в предложении. Это называется **парсингом зависимостей**.

Конечная цель этого шага – построение дерева, в котором каждый токен имеет единственного родителя. Корнем может быть главный глагол.

После первого подхода мы имеем следующую схему:



Но нужно не только определить родителя, но и установить тип связи между двумя словами.

Это дерево парсинга демонстрирует, что главный субъект предложения – это существительное «**London**». Между ним и «**capital**» существует связь «**be**». Вот так мы узнаем, что Лондон – это столица!

Шаг 6b. Поиск групп существительных (необязательный)

Сейчас мы рассматриваем каждое слово в нашем предложении как отдельную сущность. Но иногда имеет смысл сгруппировать токены, которые относятся к одной и той же идее или вещи. Мы можем использовать полученное дерево парсинга, чтобы автоматически объединить такие слова.

Например, вместо этого:



Можно получить такой результат:



Это необязательный шаг. Но часто это быстрый и удобный способ упростить предложение, если вместо максимально подробной информации о словах мы стремимся извлечь законченные идеи.

Шаг 7. Распознавание именованных сущностей (Named Entity Recognition, NER)

В нашем предложении присутствуют следующие существительные:

London is the **capital** and most populous **city** of **England** and the **United Kingdom**.

Некоторые из них обозначают реальные вещи. Например, «**London**», «**England**» и «**United Kingdom**» – это географические объекты. При помощи NLP можно автоматически извлекать из документа список реальных объектов.

Цель распознавания именованных сущностей – обнаружить такие существительные и связать их с реальными концепциями. После обработки каждого токена **NER-моделью** наше предложение будет выглядеть вот так:

London GPE is the capital and most populous city of **England** GPE and the **United Kingdom** GPE .

GPE – Geo-Political Entity

Большинство NER-моделей распознают следующие типы объектов:

- имена людей;
- названия компаний;
- географические обозначения
- физические обозначения
- политические обозначения;
- продукты;
- даты и время;
- денежные суммы;
- события.

Так как **NER модели** позволяют легко извлекать из сплошного текста структурированные данные, они очень активно используются в разных областях.

Шаг 8. Разрешение кореференции

В английском очень **много местоимений** – слов вроде **he, she, it**. Это сокращения, которыми мы заменяем на письме настоящие имена и названия. Человек может проследить взаимосвязь этих слов от предложения к предложению, основываясь на контексте. Но NLP-модель не знает о том, что означают местоимения, ведь она рассматривает всего одно предложение за раз.

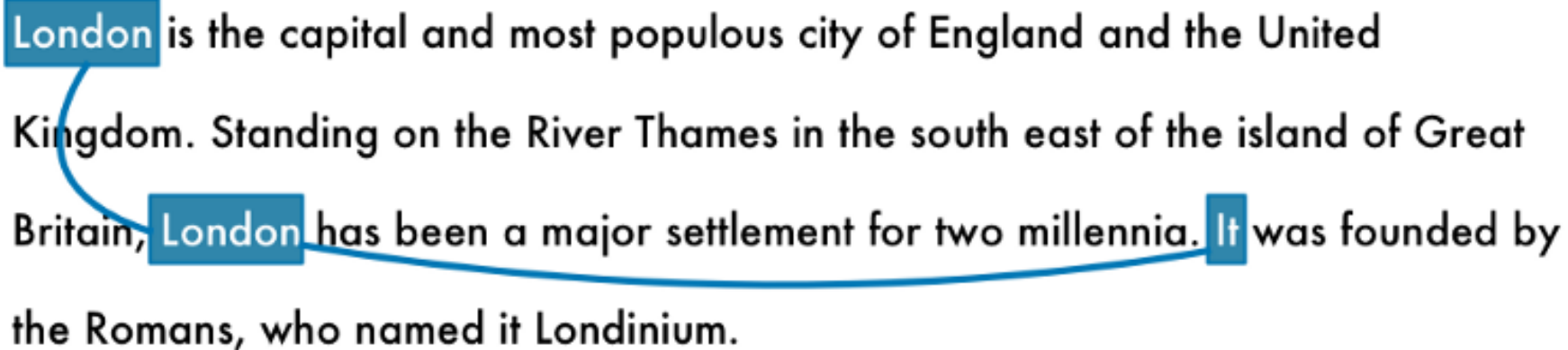
Давайте посмотрим на третью фразу в нашем документе:

It was founded by the Romans, who named it Londinium.

КОРЕФЕРЕНЦИЯ (англ. coreference) – вид текстовой или синтаксической связности, при которой две или более номинативных (именных) групп называют один и тот же объект (референт).

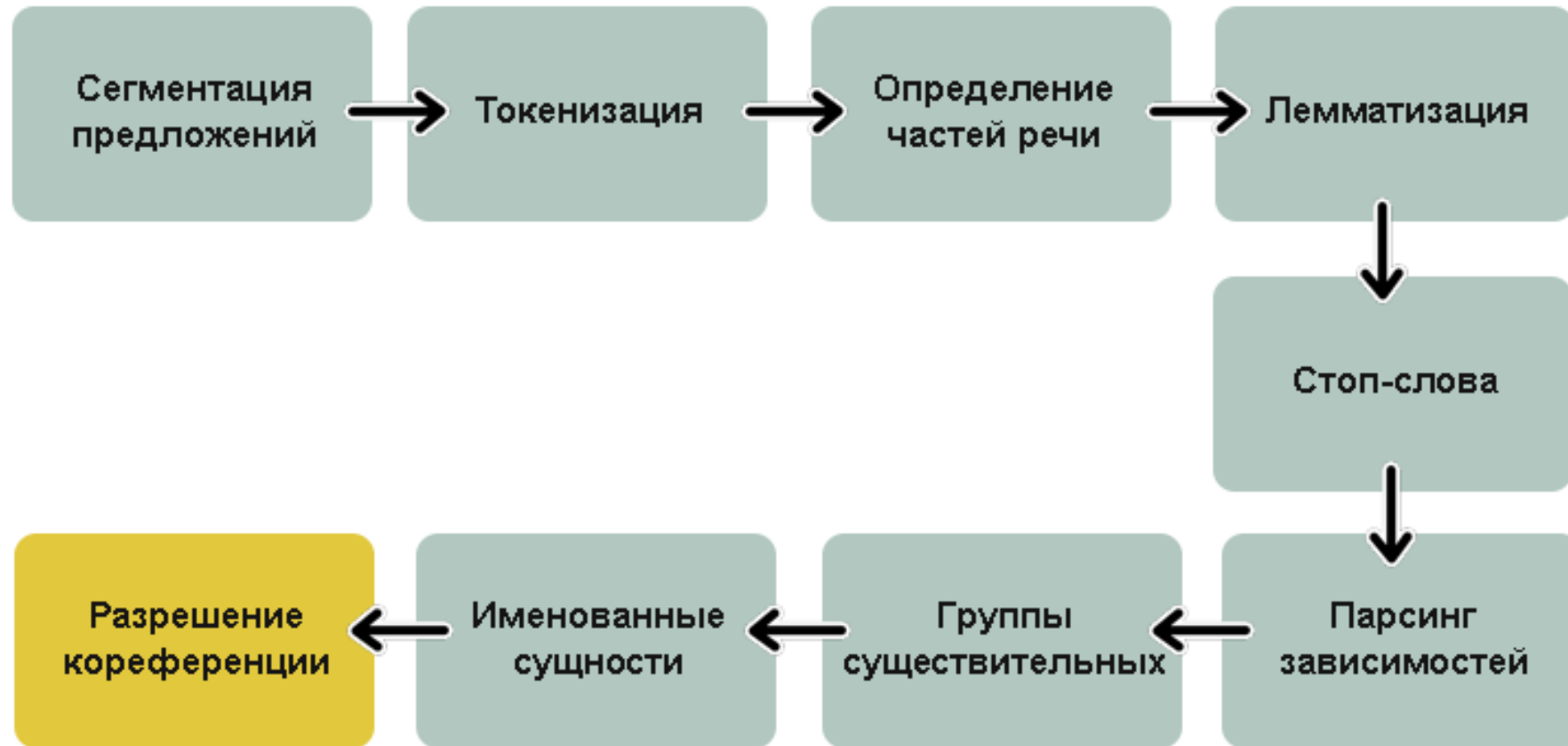
В процессе чтения мы догадаемся, что «**это**» не что иное как Лондон. Разрешением кореференции называется **отслеживание местоимений в предложениях** с целью выбрать все слова, относящиеся к одной сущности. Вот результат обработки документа для слова «**London**»:

London is the capital and most populous city of England and the United Kingdom. Standing on the River Thames in the south east of the island of Great Britain, **London** has been a major settlement for two millennia. **It** was founded by the Romans, who named it Londinium.



Разрешение кореференции – один из самых трудных шагов в нашем пайплайне. В области глубокого обучения уже появились способы его реализации, они достаточно точны, но все еще не совершенны.

Резюмирующая схема рассмотренного паплайна выглядит так:



Это стандартные этапы обычного NLP-паплайна, но в зависимости от конечной цели проекта и особенностей реализации модели, некоторые из них можно пропускать или менять местами.

Например, **sраСу** производит сегментацию позже, используя для нее результаты парсинга зависимостей.

Выделение именованных сущностей (Named Entity Recognition, NER)

Задача NER – выделить спаны сущностей в тексте (спан – непрерывный фрагмент текста). Допустим, есть новостной текст, и мы хотим выделить в нем сущности (некоторый заранее зафиксированный набор — например, персоны, локации, организации, даты и так далее). Задача NER – понять, что участок текста “1 января 1997 года” является датой, а “Кофи Аннан” – персоной.

NER-системы не просто просматривают словари. Они анализируют контекст токена в предложении и используют статистические модели, чтобы угадать какой объект он представляет. Хорошие NER-системы способны отличить актрису Brooklyn Decker от города Brooklyn.

Так как NER модели позволяют легко извлекать из сплошного текста структурированные данные, они очень активно используются в разных областях (след. слайд).

Извлечение именованных сущностей (Named Entity Recognition, NER) — это шаг в сторону “понимания” текста. Это может как иметь самостоятельную ценность, так и помочь лучше решать другие задачи NLP.

- **Структуризация неструктурированных данных.** Например, есть какой-то текст (или набор текстов), и данные из него нужно ввести в базу данных (таблицу). Классические именованные сущности могут соответствовать строкам такой таблицы или же служить содержанием каких-то ячеек.
- **Локализация важных фрагментов текста.** Например, можем выделить только те абзацы, где встречаются сущности какого-то определенного типа, а потом работать только с ними.
- **Сущности — это жесткие коллокации** и действия с ними нужно совершать как с единым блоком. Например, переводить название сущности единым куском, а не разбить на несколько не связанных фрагментов.
- Определение коллокации полезно для **синтаксического парсинга**.
- **Разрешение кореференции.** Например, пусть мы хотим проанализировать текст “Прискакал Чарминг на белом коне. Принцесса выбежала ему навстречу и поцеловала его”. Если мы выделили на слове “Чарминг” сущность Персона, то машина сможет намного легче понять, что принцесса, скорее всего, поцеловала не коня, а принца Чарминга.
- **При построении вопросно-ответных систем.** Если задать в вашем любимом поисковике вопрос «Кто играл роль Дарта Вейдера в фильме “Империя наносит ответный удар”», то с большой вероятностью вы получите верный ответ. Это делается как раз с помощью выделения именованных сущностей: выделяем сущности (фильм, роль и т. п.), понимаем, что нас спрашивают, и дальше ищем ответ в базе данных.

Коллокация — словосочетание, имеющее признаки синтаксически и семантически целостной единицы

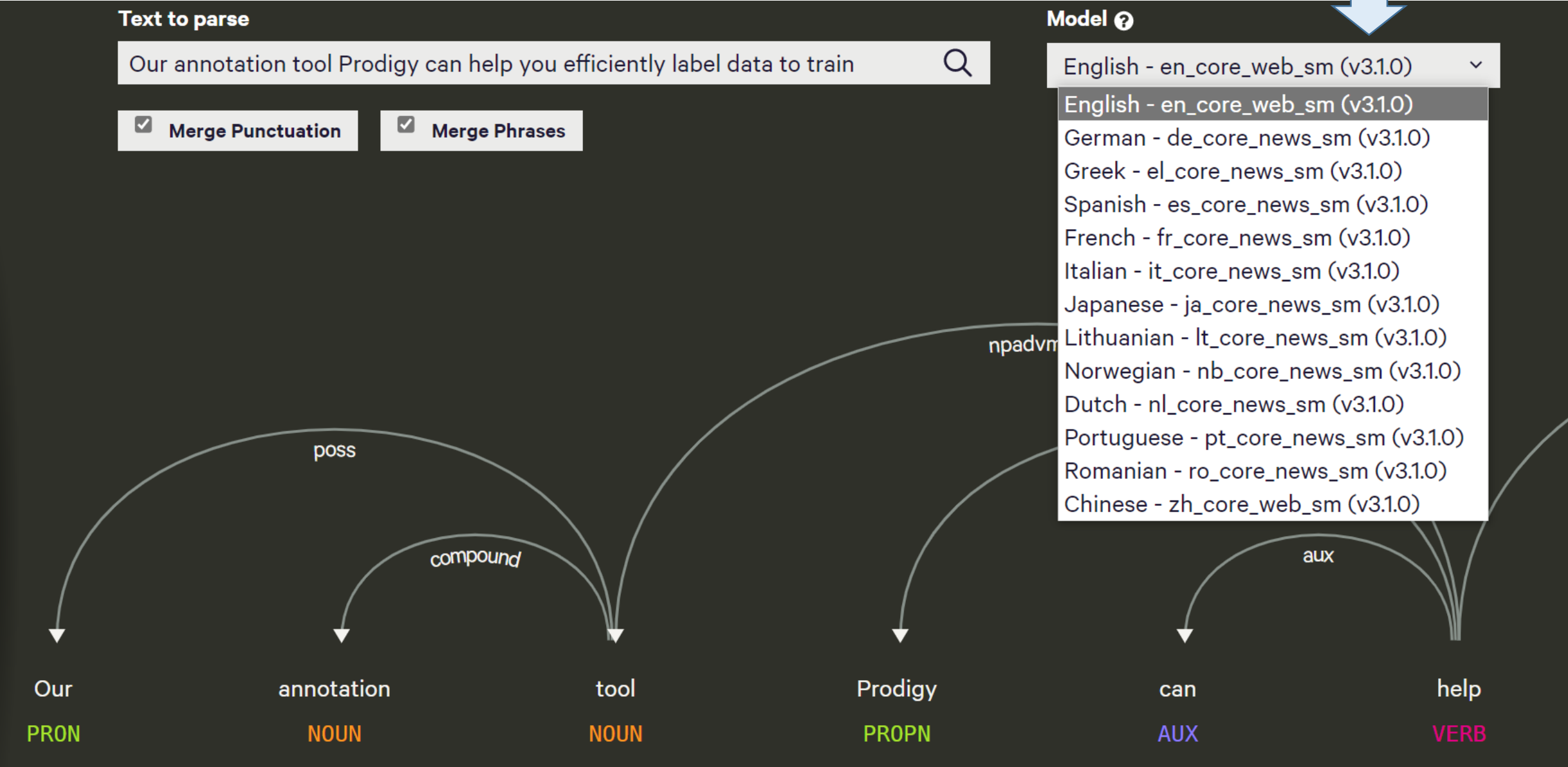
Инструменты для NLP

NLTK (Natural Language ToolKit) — самая известная NLP библиотека, созданная исследователями в данной области. Она популярна в академических кругах и в основном используется для обучения или создания различных методов обработки используя базовые инструменты, которые NLTK предоставляет в огромном количестве и не всегда самые лучшие. Так же она довольно медленная в силу того, что написана полностью на Python и работает со строками.

SpaCy — в каком-то смысле противоположность NLTK. Она значительно быстрее, так как она написана на Cython и работает с объектами. SpaCy предоставляет в основном лучшие инструменты для решения конкретной задачи. Она — пумру из мира NLP.

В целом, **SpaCy** с её предобученными моделями, скоростью, удобным API и абстракцией гораздо **лучше подходит для разработчиков, создающих готовые решения**, а **NLTK** с огромным числом инструментов и возможностью городить любые огороды — **для исследователей и студентов**.

Если кратко, то SpaCy может примерно всё то же самое, что и NLTK и их аналоги, но быстрее и точнее.



[displaCy Named Entity Visualizer · Explosion](#)

displaCy Named Entity Visualizer

When Sebastian Thrun started working on self-driving cars at Google in 2007, few people outside of the company took him seriously. “I can tell you very senior CEOs of major American car companies would shake my hand and turn away because I wasn’t worth talking to,” said Thrun, now the co-founder and CEO of online higher



Model ?

English - en_core_web_sm (v3.1.0)

Entity labels (select all)

☒ PERSON	☒ NORP	☒ ORG	☒ GPE	☒ LOC
☒ PRODUCT	☐ EVENT	☐ WORK OF ART		
☐ LANGUAGE	☒ DATE	☐ TIME	☐ PERCENT	
☐ MONEY	☐ QUANTITY	☐ ORDINAL	☐ CARDINAL	

When **Sebastian Thrun** PERSON started working on self-driving cars at **Google** ORG in **2007** DATE, few people outside of the company took him seriously. “I can tell you very senior CEOs of major **American** NORP car companies would shake my hand and turn away because I wasn’t worth talking to,” said **Thrun** GPE, now the co-founder and CEO of online higher education startup Udacity, in an interview with Recode **earlier this week** DATE.

A little **less than a decade later** DATE, dozens of self-driving startups have cropped up while automakers around the world clamor, wallet in hand, to secure their place in the fast-moving world of fully automated

```
import spacy
from spacy.lang.en.examples import sentences
```

Пример кода

```
# Загрузка английской NLP-модели
nlp = spacy.load("en_core_web_sm")
```

```
# Текст для анализа
```

```
text = "London is the capital and most populous city of England and the United Kingdom."
```

```
# Парсинг текста с помощью spaCy. Эта команда запускает целый конвейер
```

```
doc = nlp(text)
```

```
print(doc.text)
```

```
# текст токена, начальная форма, часть речи, является ли стоп-словом
```

```
for token in doc:
    print(token.text, token.lemma_, token.pos_, token.is_stop)
```

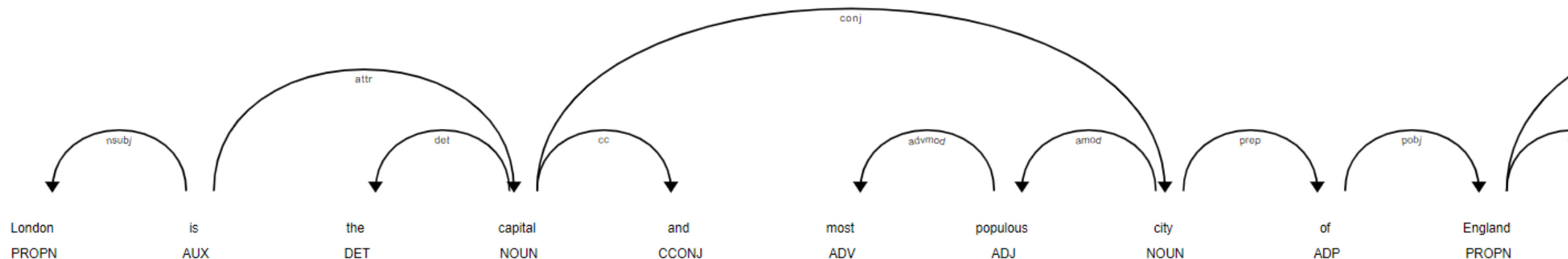
```
London London PROPN False
is be AUX True
the the DET True
capital capital NOUN False
and and CONJ True
most most ADV True
populous populous ADJ False
city city NOUN False
of of ADP True
England England PROPN False
and and CONJ True
the the DET True
United United PROPN False
Kingdom Kingdom PROPN False
```

```
# Построение дерева зависимостей  
 #(текст токена, тип зависимости(согласно Universal Dependency), корневое слово)  
  
for token in doc:  
    print(token.text, token.dep_, token.head)
```

```
London nsubj is  
is ROOT is  
the det capital  
capital attr is  
and cc capital  
most advmod populous  
populous amod city  
city conj capital  
of prep city  
England pobj of  
and cc England  
the det Kingdom  
United compound Kingdom  
Kingdom conj England  
. punct is
```



```
# визуализации дерева зависимостей, а также распознанных сущностей
from spacy import displacy
displacy.render(doc, style='dep', jupyter=True) # первые 10 doc[:10]
```



```
# для расшифровки названий тегов можно воспользоваться функцией explain:
print(spacy.explain("AUX"))
print(spacy.explain("CCONJ"))
```

auxiliary

coordinating conjunction

Распознавание именованных сущностей (Named Entity Recognition, NER)

```
doc2=nlp("BBC correspondent Stive Bob has visited the Ukraine in 2021.")

for ent in doc2.ents:
    print(ent.text, ent.label_)
displacy.render(doc2, style='ent', jupyter=True)
```

BBC ORG

Stive Bob PERSON

Ukraine GPE

2021 DATE

BBC **ORG** correspondent Stive Bob **PERSON** has visited the Ukraine **GPE** in 2021 **DATE** .

```
print(spacy.explain("GPE"))
```

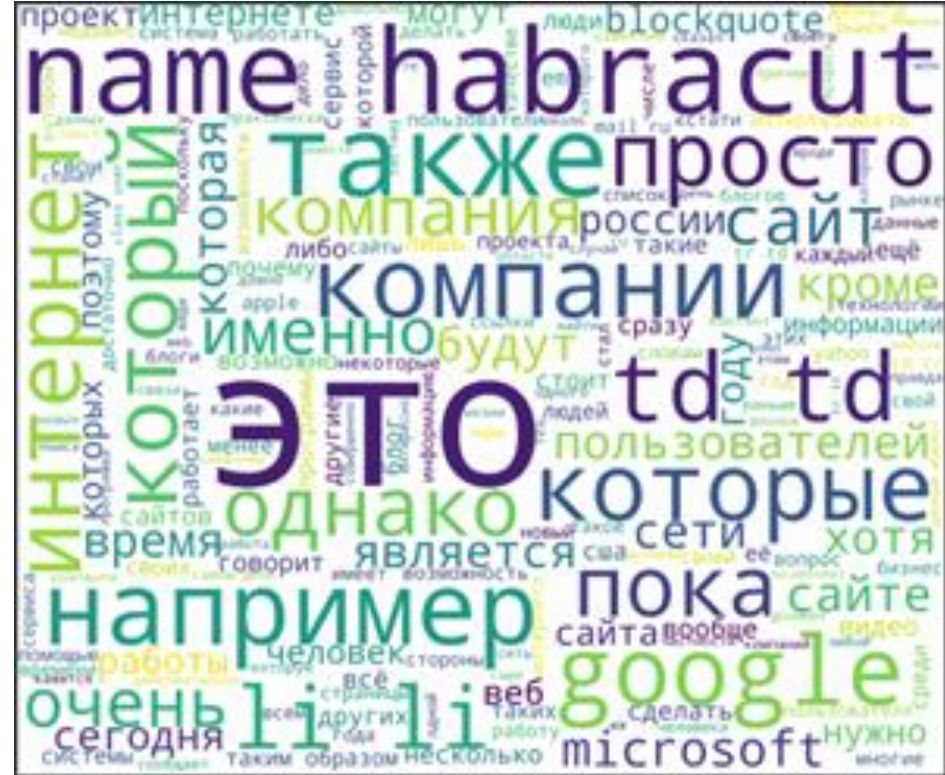
Countries, cities, states

Облако слов на Python

```
from wordcloud import WordCloud
```



Для необработанного набора данных «облако слов» содержит **243024** уникальных слова и выглядит так:



После очистки текстов от «стоп-слов» и знаков пунктуации количество уникальных слов снизилось до **142253, а «облако слов» стало более осмысленным:**

Очистка текста для облака слов

- **Удаляем специальные символы** : специальные символы, такие как двойные кавычки « \ n », должны быть удалены из текста, поскольку мы не ожидаем от них какой-либо предсказательной силы.
- **Верхний регистр / нижний регистр**: «Книга» и «книга» будут одним и тем же словом и будут иметь одинаковую предсказательную силу.
- **Знаки препинания**: такие символы, как «?», «!», «;» удаляем.
- **Притяжательные местоимения**: можно ожидать, что «Tramp» и «Tramp's» обладают одинаковой предсказательной силой.
- **Стемминг или лемматизация**: стемминг - это процесс сокращения производных слов до их корня. Лемматизация - это процесс сведения слова к его лемме. Основное различие между обоими методами заключается в том, что лемматизация предоставляет существующие слова, тогда как выделение корня дает корень, который может не быть существующим словом. Мы использовали лемматизатор на основе WordNet.



Классификация текстов и анализ тональности

Классификация текстов — соотнесение документа к одной из нескольких категорий. Применяется в:

- разделения веб страниц и сайтов по тематическим каталогам;
- борьбы со спамом;
- определение языка текста и т.д.

Анализ тональности текста (англ. Sentiment analysis) — определение эмоциональной окраски (тональности) текста. В общем случае, задача анализа тональности текста эквивалентна задаче классификации текста, где категориями текстов могут быть тональные оценки. Примеры тональных оценок:

- позитивная;
- негативная;
- нейтральная.

Векторное представление текста

Для того, что бы компьютер мог классифицировать текст, текст нужно представить в виде чисел.

Векторизация текста — это процесс преобразования текста в числовые векторы, понятные машинам для обработки в задачах машинного обучения. В машинном обучении векторизация является шагом в извлечении признаков.

Простые методы векторизации:

- ✓ Мешок слов
- ✓ TF-IDF

Недостаток этих методов — теряется информация о смысле слова.

Bag of words (BOW)

	the	red	dog	cat	eats	food
1. the red dog →	1	1	1	0	0	0
2. cat eats dog →	0	0	1	1	1	0
3. dog eats food →	0	0	1	0	1	1
4. red cat eats →	0	1	0	1	1	0

Problems:

- No information about words order
- Word vectors are huge and very sparse
- Word vectors are not normalized
- Same words can take different forms

Используя алгоритмы вроде Bag of Words, мы теряем порядок слов в тексте, а значит, тексты "i have no cows" и "no, i have cows" будут идентичными после векторизации, хотя и противоположными семантически.

Bag of words

```
from sklearn.feature_extraction.text import CountVectorizer

corpus = [
    'All my cats in a row',
    'When my cat sits down, she looks like a Furby toy!',
    'The cat from outer space',
    'Sunshine loves to sit like this for some reason.'
]

# Векторизация предложений при помощи "мешка слов"
vectorizer = CountVectorizer()
print( vectorizer.fit_transform(corpus).todense() )
```

! Не учитывает
порядок слов

```
[[1 0 1 0 0 0 0 1 0 0 0 1 0 0 1 0 0 0 0 0 0 0 0 0]
 [0 1 0 1 0 0 1 0 1 1 0 1 0 0 0 1 0 1 0 0 0 0 0 1]
 [0 1 0 0 0 1 0 0 0 0 0 0 1 0 0 0 0 0 0 1 0 1 0 0]
 [0 0 0 0 1 0 0 0 1 0 1 0 0 1 0 0 1 0 1 0 1 1 0 0]]
```

Длина вектора 26

TF-IDF (от англ. TF — term frequency, IDF — inverse document frequency)

TF (Частота слова): Насколько часто слово встречается в конкретном документе.

IDF (Обратная частота документа): Уникальность слова в коллекции; снижает вес часто встречающихся слов (например, «и», «в», «что»).

TF-IDF — мера, используемая для оценки важности слова в контексте документа, являющегося частью коллекции документов.

Вес некоторого слова пропорционален частоте употребления этого слова в документе и обратно пропорционален частоте употребления слова во всех документах коллекции.

```
from sklearn.feature_extraction.text
import TfidfVectorizer
```

Sentence A: The car is driven on the road.

Sentence B: The truck is driven on the highway.

Word	TF		IDF	TF * IDF	
	A	B		A	B
The	1/7	1/7	$\log(2/2)=0$	0	0
Car	1/7	0	$\log(2/1)=0.3$	0.043	0
Truck	0	1/7	$\log(2/1)=0.3$	0	0.043
Is	1/7	1/7	$\log(2/2)=0$	0	0
Driven	1/7	1/7	$\log(2/2)=0$	0	0
On	1/7	1/7	$\log(2/2)=0$	0	0
The	1/7	1/7	$\log(2/2)=0$	0	0
Road	1/7	0	$\log(2/1)=0.3$	0.043	0
Highway	0	1/7	$\log(2/1)=0.3$	0	0.043

Вес слова

ИТАК:

Векторизация –цифровое представление текста для обработки его методами машинного обучения.

Простые методы векторизации:

- Кодирование символов/слов числами
- Мешок слов
- TF-IDF

Преимущества:

- Интуитивно понятные методы
- Легко реализуются

Недостатки:

- Не учитывается порядок слов, это может привести к потере смысла фразы.
- Не эффективно используется память, так как для каждого слова создается вектор, длина которого равна общему количеству слов в словаре, и при этом большая часть вектора равна нулю.

Альтернатива:

Word Embedding (метод вложения слов)

Метод вложения слов - это метод обработки естественного языка, который используется для представления слов в среде глубокого обучения.

Основное преимущество использования вложения слов состоит в том, что **ОНО позволяет группировать слова схожего контекста, а разные слова располагать далеко друг от друга.** Это делается с помощью матрицы вложения. Сходство двух слов можно найти с помощью косинусного сходства.

Матрица вложения:

	King	Queen	Princess	Boy
Royal	0,99	0,99	0,99	0,01
Male	0,99	0,02	0,01	0,98
Female	0,02	0,99	0,99	0,01
Age	0,7	0,6	0,1	0,2

Два наиболее популярных алгоритма вложения слов **WORD2VEC**, и **GLOVE** (реализованы в Gensim) позволяют представлять слово в виде вектора (часто называемого вложением). Эти алгоритмы выявляют семантическое сходство слов и отражают различные аспекты значения слова .

Они используются во многих приложениях НЛП, таких как анализ настроений, кластеризация документов, ответы на вопросы, обнаружение перефразирования и так далее. Предварительно обученные модели для обоих этих внедрений легко доступны, и их легко включить в код Python.

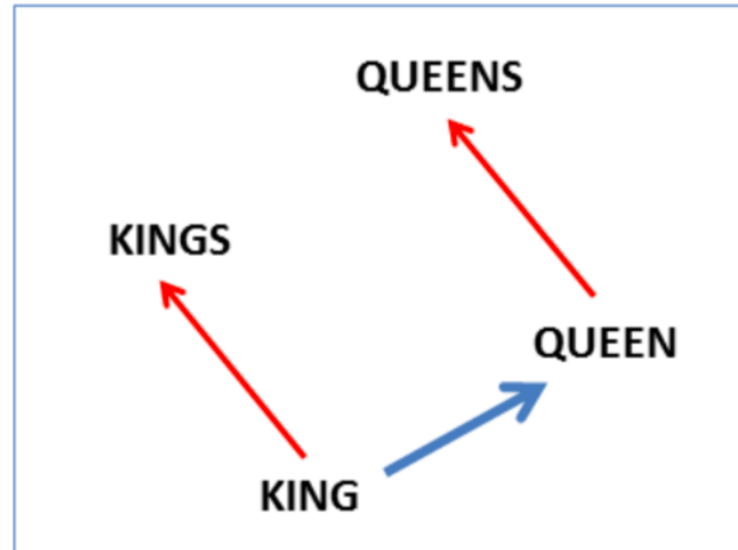
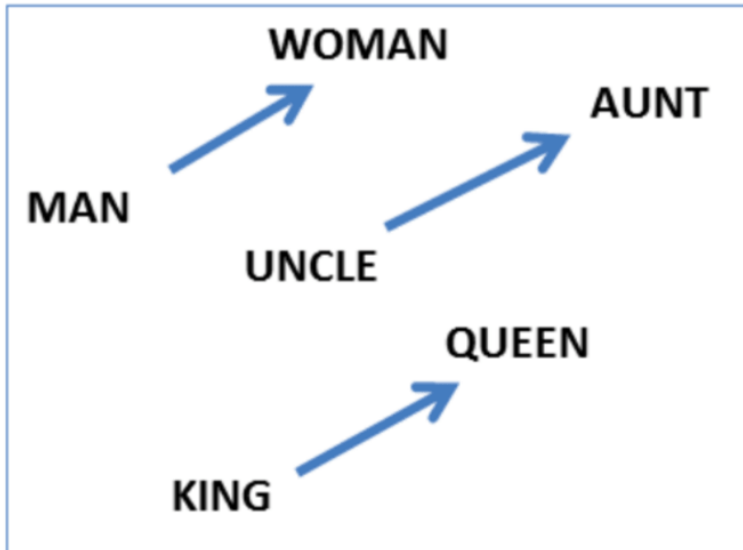
Концепция вложений (embeddings) — одна из самых замечательных идей в машинном обучении. Если вы когда-нибудь использовали Siri, Google Assistant, Alexa, Google Translate или даже клавиатуру смартфона с предсказанием следующего слова, то уже работали с моделью обработки естественного языка на основе вложений. За последние десятилетия произошло значительное развитие этой концепции для нейронных моделей (последние разработки включают контекстуализированные вложения слов в передовых моделях, таких как BERT и GPT).

Линейная структура

Удивительно то, что многие семантические и синтаксические отношения между словами (почти) линейны в векторном пространстве слов. Например, разница между королем и королевой (почти) такая же, как между мужчиной и женщиной. Или слово, которое похоже на королеву в том же смысле, что короли похожи на короля, оказывается ферзями.

semantic: $v(\text{king}) - v(\text{man}) + v(\text{woman}) \approx v(\text{queen})$

syntactic: $v(\text{kings}) - v(\text{king}) + v(\text{queen}) \approx v(\text{queens})$



Пример мужчина-женщина $\approx$ король-королева самый популярный, но есть и другие интересные закономерности



